

Natural Language Processing

DATE

Page No.

Year

I

In early 1900s,

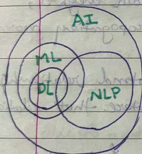
1913

Professor named Ferdinand de Saussure died, and in the process he deprived the world for the concept of "Language as a Science".

Before that ppl used to study language as a literature. He has proposed that languages is not just a way of communication. There is science behind that which is involved.

Somehow, if we will be able to teach a machine that how to decode all of those word-to-word relations, word-to-sentences relations, character to character relations.

In short machine will also able to learn relationship which as human we are able to do.



AI is not a technology, AI is final application that we used to build bas on ML, DL & NLP.

In the year 2011, Apple's Siri became the world's first successful NLP/AI assistants to be useful by general consumers.

They also in 2018-19 China has introduced world's first female AI News anchor.

Usage of NLP :- Search, Autocomplete

- Search Autoorrect

- Machine Translation

- Messenger Bots

ANN : It won't be actual neurons, axons, dendroids. It will be having almost similar kind of net architecture. net. will be trying to understand all those info. able to find out relation between dataset, based on that it will try to give me a solution.

Basic building block inside any ANN is called as perceptron.

- Perceptron: It's entity / unit, which will try to take some data set as input. It could be possible that we can implement some mathematical fun and then it can try to give solution.

→ Single perceptron can't build a relationship. If i will have a bag of perceptrons then for sure i will be able to form any number of relationships in between inputs.

* In CNN Model:

Whenever we are trying to train a model, it will try to take a data as input, then it will try to train weights, it will also update the weights in backward propagation accordingly.

Basically, in CNN model, it will just try to understand relationship, try to understand weights and just try to store that relation.

Suppose, we have images of dogs and cats. By finding out common entity / pattern in between dogs and cats. It was trying to adjust some weights for two classes.

i.e

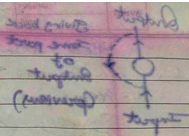
If we it is able to find out this kind of pixels then it is Dog and that kind of pixels then it is cat.

→ But it never remembers that it has done last time. Means Based on memory it's not able to give any kind of solution.

So, That is the problem we are facing with respect to VGG16, SSD, MobileNet, ResNet 50, GoogLeNet, Darknet (Variants of CNN).

RNN :- ot - output, and, might
searching other correct labam

→ RNN -



(1) :- labam output - ot - output (1)

If we are working with speech etc, text, city crn's models, are not sufficient enough?

Let us are trying to build Ques-Ans. (chatbot) kind of model or system, so whenever i will ask, it should be able to give an answer.

Let, 'Bull is going high'. can be animal

→ We know we are talking about stock market, how 3rd person will know?

If i say,

"In Fintech huge investment is done. So Bull is high...!"
Now every one can understand.

Here it is not going to take just current input but it will be able to understand previous input and output that i will give afterversee.

→ Unless and until you are not able to understand / memorise the concept you aren't able to perform / answer task."

→ Here CNN model fails. (2)

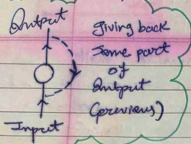
But in CNN it will try to learn Kernel = Matrix & Forward conn.

Every time we feed some inputs to CNN model, it will give output but at every new input it has no memory.

To Solve this Pb,

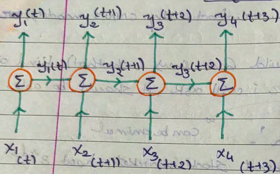
I have to store previous output. we need such kind of model which will be able to memorise previous output.

- RNN comes into picture.



Again, here Sequence-to-Sequence kind of model comes into picture.
 → I will try to build Sequential model.

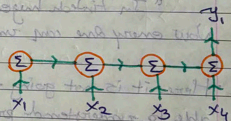
(1) Sequence-to-Sequence Model : (Seq 2 Seq)



Where, you will give n numbers of input and you will be able to get n numbers of output.

Here, Every network is trying to receive basically two inputs, one is current input and another is input from the previous output.

(2) Sequence-to-Vector :-



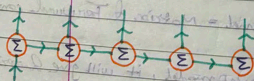
represent one LSTM architecture.

You will feed multiple inputs and will get one single output.

If I am feeding [HELL] and I want to generate next character automatically, possible.

→ As output we will get a Vector.

(3) Vector-to-Sequence :-



Here, it will be only one input and there will be multiple outputs.

4) Vector-to-Vector :



There will be only one input and one output and that will be propagate back to next time.

RNN Architecture :-

Imp

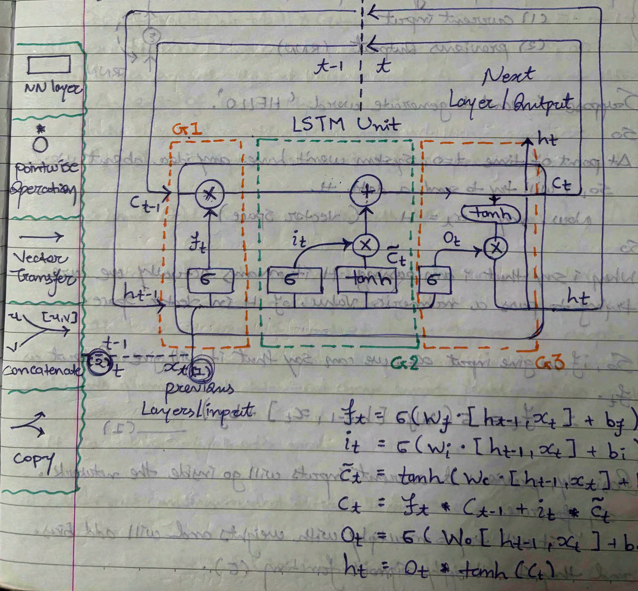
RNN-LSTM :-

Let there is word 'HELLO' and our model has to predict

If I am giving an word H. then it should give me relevant word. : HOLA, HELLO, HI, HE.

Out of these four words which it has generated, I can say that HI is more relevant one. (as compare to others).

Let's try to understand how system will be able to generate next word, how it will be able to verify, how it will be able to memorise.



G1: Forget Gate
G2: Memory Gate
G3: Final (Output) Gate.

Here, t indicates current time and $t-1$ for previous (past).

So,

at input we are feeding this particular input.

Basically,

In that architecture, we have three gates: G1, G2, G3

So, we are passing H to next. H is hidden state and x_t is input.

First Gate (G1) called as 'Forget Gate'.

It always helps to network to forget something. We may be to reduce to effect of previous learning.

So, at this point, we are passing two inputs.

As we can see, there is, every time while passing the input, we are passing two inputs,

(1) current input

(2) previous output (RNN)



Suppose, I want to generate word 'HELLO'.

So,

At point of time $t=0$, system won't have any idea about word.

So, I will try to send a data H .

Now, my input $x_t = H$ (vector space)

So,

When I say that I am passing H , it means actually we are trying to pass a numeric value of H in vector space.

So, if we give input x_t , we can say that it will give output as h_t .

$$h_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

Basically, previous & current inputs will go inside the network.

So, because

it will try to multiply with weights and will add bias and then it will apply sigmoid function (σ).

h_{t-1} = previous input
 x_t = current input

So as output we are getting $(f_t \cdot w_t + (0) \cdot w_t) = 0$

Now, in architecture of G1, we can see $\otimes$ symbol there.

Basically, w_t of g_1 is 0 as of now h_{t-1} is 0 as of now h_{t-1} is 0 .

previous input is zero as of now h_{t-1} is 0 as of now h_{t-1} is 0 .
and current input $x_t = 1$.

→ In architecture we can see one line of 'C', that is nothing but a 'Memory Channel'.

So, if I am starting my learning for a first time, then for sure past memory will be zero.

$$\text{i.e. } C_{t-1} = 0.$$

$$\text{So, } f_t = \sigma(H \cdot W_t + b) \times C_{t-1}$$
$$f_t = 0$$

As of now my entire G1 value is zero.

→ 'C' always tries to memorise from previous.

→ Second Gate (G2) is called as 'Memory Gate'.

In first Gate, it always try to remember or multiply with memory channel and by doing this it always make sure that what kind of context I am suppose to adopt.

As of now still $h_{t-1} = 0$ | Some input from previous will enter into G2 gate.

$$\text{i.e. } h_{t-1} = 0 \text{ and } x_t = 1$$

As we can see in architecture H and 0 will go in σ as well as tanh.

So,

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2)$$

i.e. $\sigma(w_1 \cdot c_0 + w_2 \cdot H) + b$ Total dataset

That is the total input which I am going to give.

What input I am supposed to allow you to pass? So that will be decide by tanh.

Now, we know that tanh will always gives result betw (-1, +1).

So,

We have total data-set, but how much it should allow to pass to the network.

So,

I will try to multiply total data with some number between -1 to +1.

So, I have sent the data, unless and until it won't be able to memorise it, how it will be able to understand the context.

So, here I will try to memorise it and this is where data is going inside that particular channels. $(\tilde{c}_x)$

Then we will find out that they are using $\oplus$ symbol.

So,

Whenever new data you have inserted this time, so it will try to add that data to the channel.

Means, previously memory channel was having two value. and now it is adding some new learning to the memory channels.

So, basically it has memorise something about input vector of H .

It means, Suppose I am talking about something Hi, Hello, HRU? What is your name.

And they directly I am switching to RNA concept.

So from Hi-Hello to RNN, I have changed my context. → while talking

There must be someone which should adopt this concept each and every time. And this is the role and responsibility of Forget Gate.

G1: Forget Gate - Allows you to change the context based on current input & previous input

G2: Memory Gate - It will try to allow you to add new memory.

Third Gate is called as 'Output Gate'

Same it will receive input $h_{t-1}=0$ and $x_t=H$.

Then:

$$(5 + 0 \cdot 1 \cdot 1 + 0 \cdot 1 = 5) \quad O_t = \sigma(W_0[h_{t-1}, x_t] + b_0) \quad (5)$$

i.e. $5 + 1 \cdot 1 + 1 \cdot 1 = 7$

$$O_t = \sigma(W_1 \cdot h_{t-1} + W_2 \cdot x_t + b_0)$$

Here we can see that final output is not only dependent on current input (i.e. O_t)

Here

We are trying to multiply $\otimes$ current input (O_t) with data which is coming from memory also.

i.e.

$$h_t = O_t * \tanh(C_t)$$

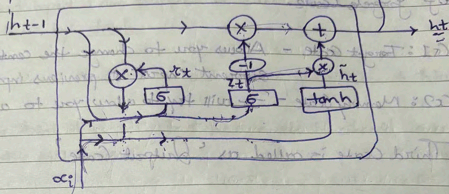
Final Output that we are receiving over here is combination of current input and previous output.

So

We can see that h_t is connected with starting part as h_{t-1} . Means system will be able to memorise and next time it will try to consider 'H' as a previous input (output).

GRU:- (2014)

GRU :- Gated Recurrent Unit



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

$$(1 * h_{t-1} + z_t * \tilde{h}_t) = h_t$$

In compare to LSTM, we will find out that there is no memory channel.

In first place it will try to calculate z_t .

As we can see in equation, it will receive two, h_{t-1} & x_t inputs.

$\therefore z_t$ will belong to 0 to 1 (range)

→ As we can see that z_t and r_t are same only.

Now, whenever we are going to calculate $\tilde{h}_t$, which is nothing but

tangent of $(r_t * h_{t-1}) \cdot W_1 + W_2 \cdot x_t$

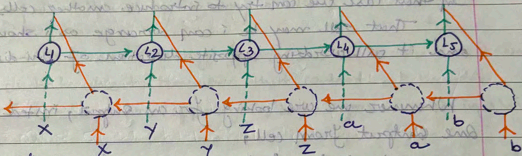
Also we can see that $(1 - z_t)$ term is giving which is nothing but

$$\text{Final output} = (1 - z_t) \cdot h_{t-1} + z_t * \tilde{h}_t \quad (\text{From the architecture})$$

Problem Statements:-

- Sentiment Analysis
- Language Translation
- Q and A (Chatbot)
- Summarisation
- Name Entity Recognition
- Sentence completion

What is the issue with the LSTM?



In case of LSTM,

we were having a cells. Basically LSTM is nothing but Seq-to-seq Sequential modelling.

In diagram we can see that (screen) is Uni-directional LSTM.

We can see that every LSTM cell is able to take an input and able to give output as well.

→ The weight of vector which I will give, it will try to flow in only one direction only.

Suppose

I have taken sentence and it is too long, and suppose I want to translate it.

"So, it will not be able to keep the entire context of the sentence as the length of my sentence is increasing."

→ So, in Uni-directional LSTM, it will try to propagate vector, then it will transfer it to the $L_2, \dots, L_5$.

And we know,

In each cell of LSTM we have forget gate & memory gate.

So, it will keep on changing the context. May be at the end of the sentence, and if sentence is too long then it won't be able to remember what it has observed from the starting. $\rightarrow$ It will start losing the context.

If we wanted to build a model which will be able to remember or understand context of long sentences.

In that case we can try to introduce another cell over there. That still may be I can arrange as I show in diagram and it will according opposite direction of uni-directional cells.

Whenever we were looking for an output, instead of giving just one output from cell,

it will try to take a combined output.

We can say that, It's a parallel cell - which we can integrate and which will be responsible to giving me a combined output.

In uni-directional LSTM if we are passing data $x_1, x_2, \dots, x_n$ then some datasets we can pass bi-directional LSTM.

But the propagation of data of bi-directional LSTM will be in reverse direction and uni-directional LSTM case it will do in forward direction.

It's not back-propagation (not updating weights), it is propagating vector in back direction, that's it.

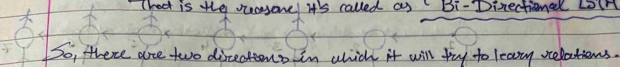
$\rightarrow$ Bi-directional LSTM will be responsible for learning in the reverse direction.

$\rightarrow$ Uni-directional LSTM will be responsible for learning in the forward direction.

Here

To give any kind of input, we are trying to consider two units and based on that two units, we are trying to take an output.

That is the reason it's called as 'Bi-Directional LSTM'.



So, there are two directions in which it will try to learn relations.

We know system doesn't understand words and grammars that is why we are passing embeddings / vectors.

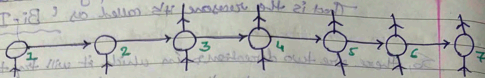
It will try to find out patterns between these vectors and based on that it will try to learn relations.

Issue was it will start learning from L1 and by the time it reaches to L5, it won't be able to remember that context.

Same things are happening in both separate case in unidirectional bi-directional.

If we integrate both of them then it will help to remember.

Encoder-Decoder is one way, input to last part of the model, we are looking for output, we are looking for output, we are looking for output.



If we look closely in this architecture, we are giving input and simultaneously we are looking for output. Let

model is looking for output ($\rightarrow$) this direction. (3rd cell)

At time t , At 3rd cell output, it is depending upon 1st & 2nd cells. But there is possibility that, Contribution of 6th number's cell is required while getting the output for 3rd cell.

In case of translation, unless and until i am not able to understand entire sentence in terms of translation, how i will be able to give output?

If i will do word-by-word trans. it may change entire meaning.

→ This kind of case it won't give us a proper result. (In both the cases Uni-directional as well bi-directional)

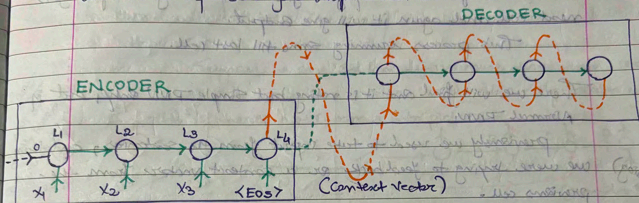
People come up idea with Encoder completely both Input as well as output, and they try to test it.

Means, try to take all input at first place, since it will be able to create input context vectors, they i will try to generate the output.

That kind of idea can give me better result. This is where Encoder-Decoder comes into the picture.

Till now we had one sequence, which was responsible for both input and output tasks. ~~sequence of words to~~

In case of Encoder and Decoder, there will be two sequence which we will have to take. One is responsible for input vector and another is responsible for output vector.



Encoder is used to encoding data and Decoder for decoding data.

Again In this net. base model will be same. We can take any base RNN model. It may be LSTM, GRU, Bi-directional.

In case of Encoder:

Just I am taking Uni-directional RNN.

Every cell is representing LSTM model where we will give input x_1, x_2, x_3 and last cell we will get $\langle \text{EOS} \rangle$ (End of String).

Which is notify that it is End of Input.

We can see it is not giving any kind of Output at that place.

In Encoder part, since it is able to understand entire context, and at last it will give Context Vector.

Now we know in Decoder also we have some cells.

First of all entire networks, Encoder and Decoder will be connected.

Also, it will try to propagate context vector to Decoder part

Whenever output it is trying to give me, it can go as input for next cell and again it will give output.

This process running same till last cell.

There we will find out it is not just Simple RNN only; but in Normal RNN

(not diag)

previously we used to take input from the sentences, and then we were trying to feed back or a context vector from the previous cell.

But here, if we observe, we are not taking input from decoder directly in decoder part,

we are trying to take output vector as input and we are using context vector from the previous one.

So, unless and until, it is not able to understand entire context vector, it won't be able to give complete output at all.

→ It is possible that I am using 5 cells in Encoder and using 4 cells in decoder.

Encoder =
 CV + MLP
 + tanh

Whenever we are talking about Image Captioning, it will try to give meaning of image. It is understanding images.

So, In Encoder As of now we were trying to use basic model as LSTM.

If we are able to replace entire cell of Encoder as CNN model. It will be able to understand which all classes are available inside the images.

Then it will try to give us word or a line based on image.

Applications:- Speech Recognition (audio-text)

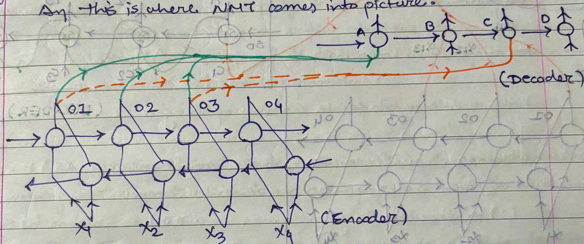
Image Descriptions Generation (Picture-Text)

Machine Translation

Neural Machine Translation:-

Suppose in one sentence, if we have more than 100 words in that case encoder-decoder isn't able to perform well. Any, this is where NMT comes into picture.

Focus
NMT by
copying learning
and obtaining



Suppose we have given inputs x_1, x_2, x_3, x_4 and it is generating output o_1, o_2, o_3, o_4 respectively.

In Decoder part to generate A, I am using output of o_1, o_2, o_3 only. Similarly to generate B, I need o_1 and o_3 so A is using that.

Meaning of this architecture is, if some decoder is trying to produce some output, it can use whatever encoder it wants to use.

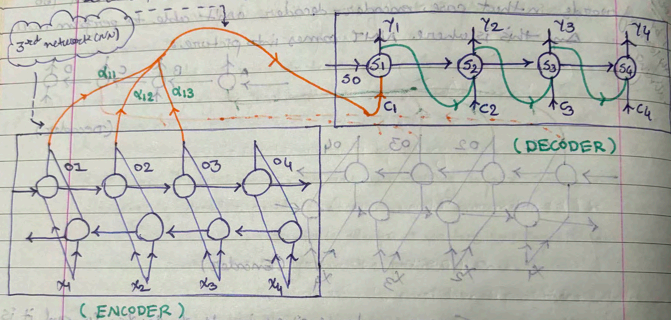
In this case we are trying to create Attention. So that it can understand that to generate output which and all words or inputs are contributing.

Because whenever we are working with translation, common kind of things are playing imp role.

201

(understand) not just simple sequence
 (People are trying to design a network which will be able to focus on what to generate and create attention by itself.)

If I have such kind of network, which will be able to figure out what in which one I am supposed to focus and which word not supposed to focus to generate some words - So in that case I will be able to do better translation.



In Encoder part we have given inputs x_1, x_2, x_3, x_4 and getting result o_1, o_2, o_3, o_4 respectively.

Similarly, in Decoder part we have cells and every cell is using input as output of its previous cells except 1st one.

Also, How it is able to create 'Attention'?

Every time keep in mind that, Suppose we are getting output y_1 . So for input for that particular cell always go through one single Neural Network from Encoder.

So, that orange line is nothing but Neural Network.

They said that let this NN learn - let that NN decide.

Although

inside Encoder we already have LSTM, which will be able to consider the context of entire sentence.

That is fine, but we are supposed to achieve - The way we are going to give input or the way we are going to focus on words or the output that we are receiving from the encoder.

So,

OK, NN will be able to pick and choose, or able to understand a relationship between the output that we are trying to give or the input that we are trying to take from the encoder.

So I will leave this things up to NN, and I will try to make this thing as learnable / trainable parameters.

- This is how we will be able to solve this problem.

So, if we are trying send context vector (Q), so context vector Q is nothing but input which is we are trying to send from encoder to decoder.

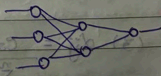
Beoz, Encoder trying to learn the relationship, Decoder is responsible to giving the output

So,

Whenever input is going to Encoder to Decoder (Q) - that will try to pass through that entire NN (orange).

We know,

if we are trying to design very simple NN, there will be inputs, they weights and biases, they output.



- where weights are nothing but trying to find out relations between inputs and outputs.

So, here we can try to replace this things with NN and we try to define ~~the~~ over there.
weights

There α is nothing but weight parameters.

is $\rightarrow$ Summation of all the inputs & respective weights

$$C_i = \sum_{j=1}^n (x_{ij} \cdot w_{ij})$$

$\therefore \sum_{i=1}^n C_i = 1$

So it will take all the inputs into neural networks and then it will try to generate one context vector.

Means at a time it will focus on all the inputs then it will try to learn that among all those inputs find encoder part on which I am supposed to focus are not supposed to focus.

This is something which is called as 'Attention'.

$\rightarrow$ We are trying to create a focus by using NN.

$\rightarrow$ There are some restrictions on weight parameter (α):

(1) α should always be positive. ($\alpha \geq 0$)

$\rightarrow$ We are always supposed to consider a positive relation.

(2) Summation of all the α s should equal to 1 because of the probability factor.

$$\therefore \sum \alpha_{ij} = 1$$

$$(3) \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^n \exp(e_{ik})}$$

$\rightarrow$ where

e_{ij} = activation (previous input, current input that we have that we are given able to get)

$$e_{ij} = a(S_{i-1}, O_i)$$

That Network will be the function of output that i am trying to get from each & every cell, previous output that it has already given, whatever word that it has produced.

The current output that i am able to get from the encoder.

So, it will try to learn, what is the output that i have given last time and what is the output that i am getting from the encoder network.

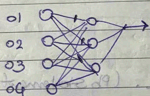
So,

previous context vector and current context vector (Encoder).

→ This is called as 'eij'.

→ eij is called 'Attention Function'.

So NN is responsible to create that particular attention / focus. (eij)



In NN, 01, 02, 03, 04 are output of Encoder part and it is trying to find out weights which is involving the most to get output (similar output) as it was getting from previous context.

Also we know that sum of all the weights are 1. it is convenient to know which one is contributing the most.

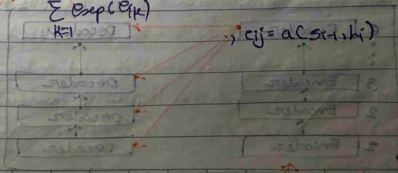
So if some words are not contributing then it will decrease the weights from there and this is how it is managing.

This is how it will be able to understand attention automatically.

But we were defining weights randomly in nn. Here to define the weights they have used $\exp(e_{ij})$.

$$\sum_{k=1}^n \exp(e_{ik})$$

$$c_{ij} = a(c_{i-1}, h_i)$$



* Attention Neural Network Self Attention :-

Encoder and Decoder was overall architecture. The Base model which we were using that was RNN.

If we are trying to change base model inside Encoder and base model inside Decoder - That is called as 'Transformer'.

→ Again pb is same - Translate one language to another language, Sentiment Analysis, Fill in the blanks, Generate sentences based on inputs, text summarization.

But how beautifully we are able to get answer that matters.

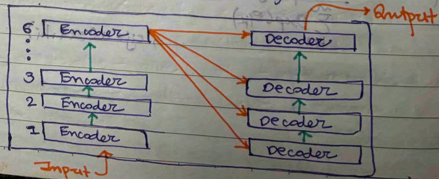
Transformer :-

→ let i have to do language Translation. (Pb Statement)

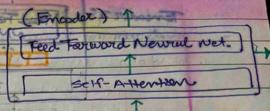
Pb: If the length of sentence is small, any base model will work well. It will be able to understand what we are trying to give input what we are expecting output.

→ If length of sentence is going to increase, it is very difficult for any network to remember then context, to understand entire grammar. To understand relationships.

→ This is where architecture of 'Transformer' comes into picture.



Now, what happens inside the Encoder:



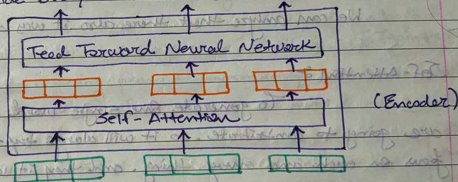
The encoders are all identical in structure (yet they do not share weights). Each one is broken down into two sub-layers as shown over here:

There will encoders and decoder. It will try to take input and will give data inside Self-Attention.

Then whenever output it will be able to get from self-attention it will try to send that output inside the feed forward net.

This process will happen 6 times and final output whenever we will get that we will pass to the decoder.

Inside Every encoder we have same this two networks.



Let i am try to give three inputs over here. It will try to convert entire three words into vector.

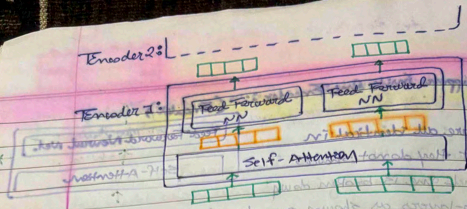
Then length of the all vectors which we are giving is 512.

Embedding will happen in bottom most encoder only.

In training instance, when we are trying to give input for very first time. So it will convert all the words into vector size of 512.

Whenever output will give this very first encoder - it will pass to next encoder directly.

→ Basically it will be length of longest sentences in our data.



Very clearly we are able to see over here is, it's not just a one Feed Forward Networks, which we are trying to send all the input.

We are having that many number of networks which is equivalent to the input that we have given.

→ In NMT also we have seen that there are diff-diff NN which is taking inputs from the Encoder and giving output to diff-diff cells in decoder.

We can analyze that there also it was diff NN same as here

Self-Attention :-

To generate one single word what all words are going to contribute. So it will always try to understand or focus on each and every thing. and then it will try to train weights. Some will be activated and some de-activated.

This concept is called as self-attention.

Let's explore what process happens inside self-attention:

Let us consider that, we are giving two inputs to encoder X1 & X2 as shown in above figure. (Size 512)

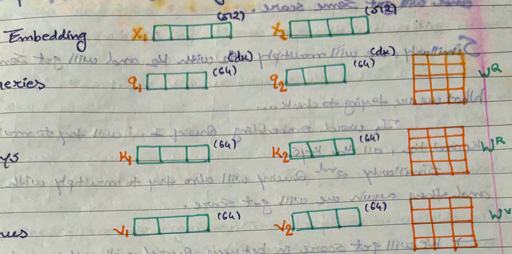
Since I will be able to give input, it will generate three things:

- (1) Queries
- (2) Keys
- (3) values

d_k = dimension of all vectors ($2, 4, 8, 16$)

Step: 1

Input sequence of Thinking Machine



This is what happens inside self-attention

Whenever we will pass data as input which is x_1 here, it will multiply with weight of Query (w_Q).

But we know in NN, that weights are random. (Trainable params)

So, q_1 is being produced by multiply with x_1 and w_Q .

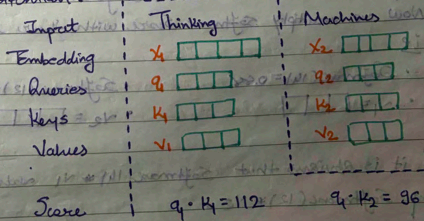
Similarly $q_2 = x_2 * w_Q$

Similarly, $x_1 * w_K = k_1$ and $x_2 * w_K = k_2$
 $x_1 * w_V = v_1$ and $x_2 * w_V = v_2$

will generate these things of certain size or dimension.

Basically we are creating these things so that it will be able to generate attention.

Step: 2



Score: $q_1 \cdot k_1 = 112$, $q_1 \cdot k_2 = 96$

In Second step we are trying to multiply q_1 with K_1 and will get some score.

Similarly, we will multiply q_1 with K_2 and will get some score.

What we are trying to check...

In word embedding Query 1, I will try to multiply with K_1 and K_2 , all the keys.

Similarly 2nd Query will also try to multiply with all keys and they again we will get score.

→ We will get score in between Query with all respective keys.

Step 3

Since we are able to generate score, I will try to divide it by $\sqrt{d_k}$. Here it is 8.

And then I will get some number.

→ That score we will try to send it inside Softmax function.

i.e. $q_1 \cdot K_1 = 112$

$q_1 \cdot K_2 = 96$

$q_1 \cdot K_1 / \sqrt{d_k} = 14$

$q_1 \cdot K_2 / \sqrt{d_k} = 12$

$\text{Softmax}(14) = 0.95$

$\text{Softmax}(12) = 0.12$

$$\frac{e^{14}}{e^{14} + e^{12}}$$

$$\frac{e^{12}}{e^{14} + e^{12}}$$

Step 4

As of now, haven't use values so far.

Now we multiply Softmax score with their respective values

i.e.

$\text{Softmax}(14) = 0.95$

$\text{Softmax}(12) = 0.12$

$v_1 = [1, 2, 3]$

$v_2 = [4, 5, 6]$

So, it is obvious that $\text{Softmax}(14) * v_1$ value will be greater than $\text{Softmax}(12) * v_2$.

Steps

Whenever calculation we have done, we got some scores. Over here with respect to q_i with all other keys (K).

after multiplication of softmax with their respective values (v).

We will get some scores.

Now we need to sum up those values called (Z).

i.e

$$Z = \begin{bmatrix} \square & \square & \square \end{bmatrix}$$

$$Z_0 = \begin{bmatrix} \square & \square & \square \end{bmatrix}$$

(If 5 inputs then we will get $Z_0, \dots, Z_5$)

This is how self-attention works in Encoder in case of Transformers.

→ In case of back-propagation, It will update the weights w^q, w^k, w^v and these are the trainable parameters.

→ We know we have generated score by $(q_i \cdot k_i)$. If score is very less then softmax will be very less. then it will try to rectify value vector.

If score is very less, it means someone is responsible for giving me this score is having some kind of problem. I have to adjust that part. (i.e. query and keys)

→ Ultimately we are generating query and key from the weight vector only.

$$\text{Attention equation} = \text{Softmax} \left(\frac{Q \times K^T}{\sqrt{d_K}} \right) \times V = Z$$

(matrix form)

Score

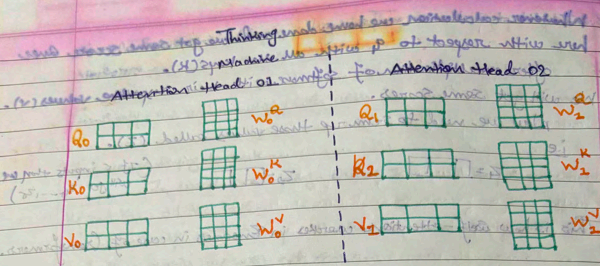
↓ Output for the word

Multi-headed Attention :-

(*)

If we are trying to increase number of heads, (num of head can be anything). No of head is nothing but number of words that we wants to focus on.

Suppose we are trying to find out relationship between multiple words at a time,



If we are working with single headed model then we will have (W^Q, W^K, W^V) . If we are working with multiple headed model then we will have that much values of weights.

→ Here we have to head so we can see that we have two diff set of weights (i.e. $Q1$).

If we will work with n number of headed model then we will have n number of sets of weights.

In multi-headed attention,

I will have multiple weights of Q, K, V .
 i.e. for single word i will have multiple values of Q, K and V .
 C.i.e. multiple numbers of Z score (output) also.

In that case i will be able to create more numbers of relations.

Then we will try to concatenate multiple Z score for final output.

Then, Multiply with a weight matrix W^O (new) that way joined jointly with the model.

Whole idea is to introduced more number of trainable parameters. Just give more amount of flexibility to network so that it will be able to adjust.

Pos. System always tries to adjust based on loss.

It will adjust on trainable parameters that is weights.

Positional Encoding:-

So, we accept that we are able to build a relations, we are able to find out attention, which one will be generated after what. All things we are able to do.

→ We are creating attention to know on which word system should focus. But network doesn't know, which word comes after what.

Network has no idea that which word it suppose to give after what.

→ Network should suppose to know which word it has to give after some particular word.

We are trying to create attention, attention simply means that generate one word, which and all words it suppose to focus.

It suppose to select those words by its significance.

But the problem is position. How it will be able to arrange the words. How it will be able to arrange positioning of words.

→ We know in language, it's fine that it is able to create words but unless and until it is not able to arrange those words it won't able to make any meaning.

- Here positional Encoding comes into picture.

We have seen that we were trying to convert those words into embeddings of size 512.

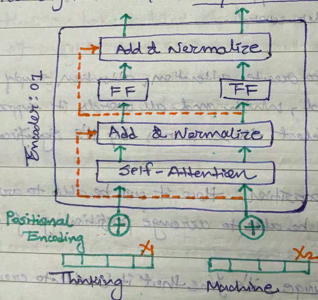
But I am suppose to attached some more information we can call as 'Meta Information' as well.

So, What are we doing here is we are trying to attach one more vector with remembering sequence as things like it which will be responsible for remembering a position of input word.

→ whole idea was instead of focusing on just relations of word it should also focus on which word is coming after which.

The Residuals:

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ff) in each encoder has a residual connection around it and is followed by a layer-normalization step.



Again, we have taken word, then done word embedding then added positional Encoding.

Then we are trying to send all the data to self-attention.

Inside that it will try to generate Q, K and V .

Then it will perform, $\text{Softmax} \left(\frac{Q \times K}{\sqrt{d_k}} \right) \cdot V$

Then depends upon significance and it will try to generate Z .
That will be output of Self-attention.

→ After that we will try to Normalize data set. (Basically z).
So that, that normalize data we can pass to the Feed forward

network. processing got tanks. network.
In FFNN, it will give some output, again we will normalize that output and we will try to pass it to next encoder.

Now, as we are able to see, $(---)$ line in network. why?

As we can see that line has been drawing from input to directly normalize layer, again. Some output to direct normalize layer.

Instead of trying to send data in attention path, what if i want send data inside ^{Add norm.} self-attention. Means we are trying to skip attention part. Similarly we are skipping FFNN part as well.

As we remember, in ResNet architecture also ppl have created residual connection / Identity connection.

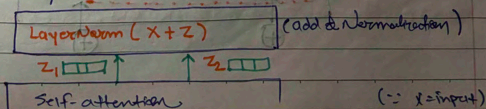
If there is significance of learning something, i will send data to a network. If there is no significance,

But inside attention i am trying to find out focus. after certain epoch, i am not suppose to, i am able to get score or that score is pretty much fine for me, i am able to adjust weights as well. They why to send data in next epoch to self-attention.

What can i do over here, is instead of sending data to next attention i can directly send data for normalization.

- That is called as 'Skip Connections'.

What does mean by Add & Normalization?



- The Decoder Side :-

The encoder start by processing the input sequence. The output of the top encoder is then transformed into a set of attention vectors k and v . These are to be used by each decoder in it's 'encoder-decoder attention' layer which helps the decoder focus on appropriate places in the input sequence.

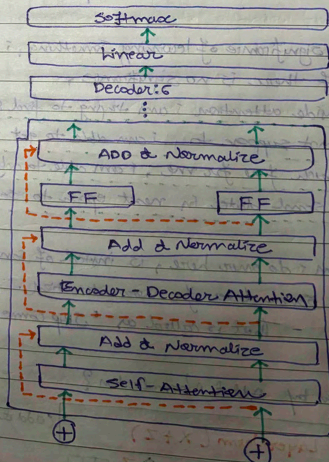
→ We will pass data to Encoder and then we have seen that at and all process will happened inside all the encoder.

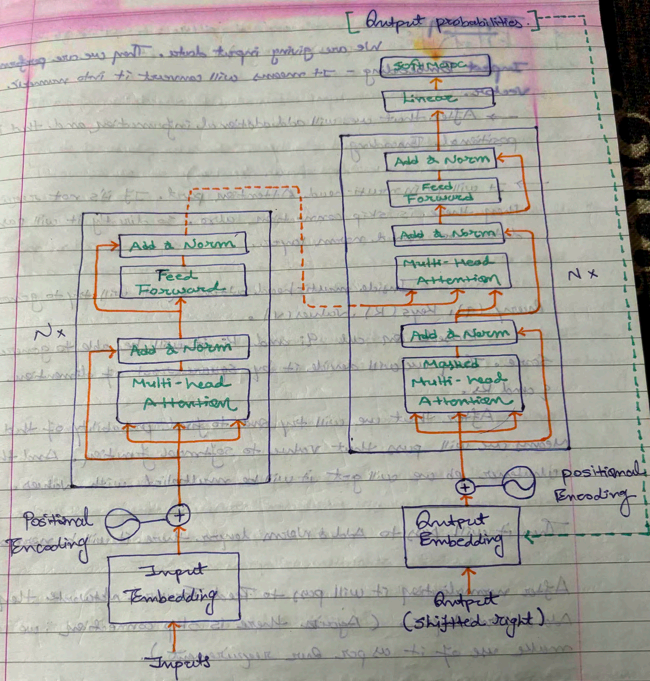
Now,

After completing last encoder part it will try to pass output in form of vector of Key and Value.

So that decoder can use k and v vector for it's 'Encoder-Decoder Attention' part inside decoder.

Decoder:





The Transformer - model architecture.

Explanation:

We are giving input data. Then we are performing Input Embedding - It means will convert it into numeric vector.

→ After that we will add additional information and that is positional Encoding.

→ It will go in multi-head Attention part. If it's not required then there is skip connection also. So directly it will pass data to add a norm layer.

Inside multi-head attention it will try to generate Query (q), Keys (K), Values (V).

Based on all q and K it will be able to generate the score. Then we will divide it by square root of dimension of q and K .

After that we will try out to find probability of that values. Means we will pass that value to softmax function. And then whatever prob we will get it will be multiplied with values.

Then it will pass to Add a Norm layer there it will be normalized.

After normalization it will pass to Feed Forward Network they again Add a Norm layer. (Again there is skip connection, we can make use of it as per our requirements).

→ This entire process of one encoder. Inside every encoder same process will happens.

At the end of Encoder we will get final output from the encoder part as Key vector and Value vectors.

That both the vectors will pass to the decoder inside Multi-head attention part. (Take very first time)

Inside multi-head, again it will perform some task.

Then it will pass to Add & Norm layer to normalize it.

Also there will be available skip connections.

Then some process, - FFN - to Add & Norm with skip conn.

This will go through every decoder part.

Then final output will go to linear part. and it will generate some logits (members).

Those logits will pass to the Softmax layer. Then based on probability it will give final output.

Since we are able to get output, next time we are going to send that outcome as well in bottom of decoder part. (Output embedding)

Some process will happen over there, positional Embeddings & all for 2nd time.

2nd time as we process further, it will use this output which is coming from Output Embedding as well. Output which is coming from Encoding part also.

This is how entire network works.

• What is 'Masked Multi-Head Attention'?

It can be used to use Dropout. Means there are were using (trying) to masked some of the weights. We were trying to hide those weights.

Let,

My name is Aakash.

I am sending that sentence as: My name is Aakash.

In that case it will try to understand what will come after name. That is called as 'Masking'.

BERT

(2019) (Google) seq2seq

→ BERT: Bidirectional Encoder Representation from Transformers

Introduction

BERT is basically an Encoder stack of transformers architecture. (We don't have decoder)

Transformer is base architecture that people have used inside BERT architecture.

Transformer is architecture of encoder-decoder network that uses a self-attention on the decoder side and attention on the encoder side.

It was 6 layers of both encoder-decoder

→ BERTBASE has 12 layers in the Encoder stack.

→ BERTLARGE has 24 layers in Encoder stack.

Two versions of BERT

These are more than the Transformer architecture describe in the original paper (6 encoder layers).

BERT architectures (BASE and LARGE) also have large feed-forward networks (768 and 1024 hidden units respectively).

And more attention heads (12 and 16 respectively) than the Transformer suggested in the original paper.

It contains 512 hidden units and 8 attention heads.

BERT-BASE contains 110M parameters while BERT-LARGE has 340M parameters.

→ GPT3, They are using 175 B parameters.

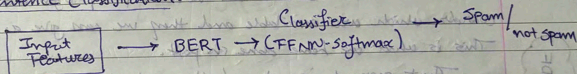
We know that these networks were not really designed for classification task. CNN: We were talking about AlexNet, VGG, GoogLeNet, ResNet... These networks were created to just perform a classification task.

Object Detection: It was using classification, as well as detection. It was regressing boundary boxes as well. These model was performing two tasks at a time.

Segmentation: It will do classification + detection + generate pixels.

→ Similarly here also they have build model - BERT, So that it can perform multiple task at a time.
i.e

1) Sentence Classification.



BERT is attention based model and it will be able to understand relation in much much better manner. It will be able to understand if class = 1, what and all words are responsible for generating class 1.

* What is Encoding?

TF-IDF:

→ TF-IDF = Term Frequency - Inverse Document Frequency

First of all to perform RNN or encoder-decoder or transformers. For that we need embeddings which will feed to model.

Ans, (1) My name is Adarsh.

(2) I work for CV. *light-Dark: 14 min*

(3) I use to take ML-PL days in weekend.

Suppose someone ask to do some kind of classification, summarization to produce new sentence, auto-complete sentence, these are NLP tasks.

→ How to produce numerical values out of text?

One simple possible way is,

Extract all the dockeys, try to find out distinct words which are available and then we can give a number. This is what we have done in past.

That approach is not good it can be bias. Bcz number that we are applying is individual magnitude not a relative magnitude trying to apply. That is the prob.

$$TF = \frac{\text{Term } i^{\text{th}} \text{ Frequency in doc } j}{\text{Total number of words in } j}$$

$$\text{IDF} = \log_2 \left(\frac{\text{Total document}}{\text{Document with } i \text{ term}} \right)$$

word embedding = $TF \times IDF$

1)	My name is Adarsh	I work for CU	use to take ML-DL
2)	0		
3)	0		

No. Document = ~ Numbers of records.

→ Total members of input channels = 3. (document)

Calculation: ~~unmarked~~ / ~~unmarked~~ $\rightarrow$ ~~unmarked~~ $\rightarrow$ ~~unmarked~~ (Embedding Generation)

First i will start with commonality by 1/10/18 2004

→ First of all, we will calculate TF.

- $TF(M_q) = \frac{(\text{How many times 'M_q' is occurring})}{(\text{Which is total num of words available in 1st doc})}$

$$TF(M_y) = \frac{1}{4} \text{ (constant)} \quad \text{--- (7)}$$

- $IDF(m_i) = \log_2 \left(\frac{\text{Total num of doc}}{\text{What is doc which is having 'm_i'}} \right)$
 $= \log_2 \left(\frac{3}{1} \right)$ (How many)

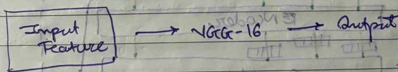
→ TF-IDF (M4) = $\left[\frac{7}{4} \times \log_2\left(\frac{3}{1}\right) \right] = 2.45$ for each energy line.

$$\rightarrow \text{TF-IDF (Name)} = \left[\frac{1}{6} \times \log_2 \left(\frac{3}{1} \right) \right]$$

② Image Tagging :

It will try to generate a tag of images. It will be able to tell the story about images. Just like humans being.

Like as we can see image & we can say that these all data which are present into that image.



It will try to give all the classes (vec-16). then if we apply a BERT model then we can see that based on that it will be able to form sentences.

③ Word Embedding Recap:

word-embedding recap: As we know TF-IDF is formula based, word-to-vector is NN based, and all...

But if we use BERT as word embedding we know BERT is
 12 layers architecture / transformers architecture
 Also more 340M+ parameters are there

④ Task specific Models :- (p, m) $\rightarrow$ $(p, m) \times T$

(i) Sentence Pair Classification Task : $(CLS + \text{Sentence} + \text{Separator} + \text{another Sentence})$ pass as input it will be able to generate class.

(ii) Single Sentence classification Task :

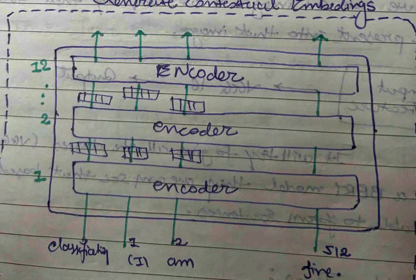
One single Sentence.

(iii) One-class task :

(iv) Single Sentence tagging task :

BERT for Feature extraction :

Generate Contextual Embeddings



* GPT₁⁰

commentary | analysis | evaluation

GPT : Generative Pre-Training

Whenever we try to do any kind of NLP Task. at that point of time, basically we need labeled data. To perform labeled classification we need huge labeled data.

→ For any supervised learning task we need labeled data.

→ Even if i am able to labeled data still i will be very difficult to generalize the model.

Let,

1> Movie is ocean.

+ve

5> I will give rating 4.9 out of 5.

+ve

2> Now it was great.

+ve

3> It was amazing.

+ve

4> It was wonderfull.

+ve

So i can express sentiment by using four diff-diff Sentiments.

There are some words which are showing positive sentiments.

We have to labeled these data, still it will be difficult to generalize these sentences. Bcoz there are many words which can indicate the sentiment.

Even though we have labeled data, generalization is very difficult over here.

→ This kind of pb we are facing in many other tasks as well like language translation, question answering, tokenization, entity classification, sentence summarization etc..

issues: - large amount of labeled data (Supervised learning)
- Generalization is very difficult.

Researchers came up with idea, ~~these are the problems that we usually facing in case of any type of NLP model~~. So why can't we build some type of model which will be able to generalize all of these things and they will try to learn by it self only.

Why are we looking for just a supervised kind of approach?
Why can't we go with un-supervised kind of approach?

[Booz - If we use un-supervised approach, in that case, labeling data is not required. In that case we may solve one problem.]

→ We can say that un-supervised learning can be serve as pre-training objective. With the help of that it will try to make clusters. It will create groups of data.

Then we can try to use supervised learning so that i will be able to fine-tune ^{un} supervised learning.

→ I will try to grouping data of similar types of semantic data. then by performing : whether that particular grouping is correct or incorrect. So may be we will be able to rectified it by using supervised learning.

Then they introduced GPT.

GPT : Means we are trying to give something based on pre-training. basically from un-supervised learning we have train something and based on that we generate something.

→ About end problem,

Here we have 5 sentences with the response, all their 5 sentences words are different. If i introduce 6th sentence for sure it won't be able to generalize those sentences.

In case of unsupervised

Maybe based on distance-based approach. Similarity based approach. It will try to bring all data together based on similarity. If it is based on unsimilarity it will try to create diff group. This is how my system will be able to solve generalise problem.

→ This is where DLT came into the picture.

As Solution, they have used Semi-Supervised learning approach. i.e. (Supervised + Unsupervised)

If we use Semi-Supervised approach where we can try to use both Supervised and unsupervised.

Unsupervised - So that it will be able to bring all similar type of data all together.

Supervised - Whatever data is brought together it will be able to validate. (Whether grouping of dataset is correct or not) Based on that we can do to create generalization.

For training supervised or unsupervised model, they have defined basically Objective function.

Objective function is nothing but a combination of loss function as well as regularization function.

Loss function: It always try to find out what is the difference between actual value and predicted value.

Regularization fun: It will try to regularize or it will try to reduce the degrees of freedom of data which is causing me an error.

Objective function for unsupervised learning is -

$$O(\text{un-supervised}) = \sum \log P(t_i | x_i)$$

$$t_i = t_1, t_2, \dots, t_n$$

→ This objective function will try to find out summation of log of probability of occurrence of t_i .
 t_i is nothing but token. Token is word of character that we are going to give as input.

→ What is the probability of token t should occur or word should occur if $t = t_i$ (Set of token is given to me).

→ K stands for window. It's length of data that we are going to pass.

→ θ is parameter of NN, on which we can perform Backward propagation.

[Let there are three clusters A, B, C. So what is the probability that word 'movie' is part of cluster A, B or C.]

→ This is how it works. (unsupervised)

Objective function for supervised learning is -

$$O(\text{supervised}) = \sum \log P(y | x_1, x_2, \dots, x_n)$$

It will try to find out, summation of log of probability of occurrence of y when x is given to me.

Simply it is trying to predict that what is the label, what is the probability of occurrence of labels when $(x_1, x_2, \dots, x_n)$ these are the features that given us.

→ In case of Sentiment Analysis,

I don't want to label huge data, I have small amount of labeled data and huge amount of unlabeled dataset.

I need to build a ~~very~~ generable model which will be able to learn everything, which will be able to give sentiment of data without ~~giving~~ giving all sort of combinations.

So what I will do in first place is,

Whatever ~~un-labeled~~ data that I have, I will try to pass those data into a unsupervised learning. Then whatever labeled data that I have, from there I will try to build a supervised ML model.

Now I have cluster of the dataset and then I have one unsupervised model. Now we know,

In case of unsupervised it will try out to find pb. of occurrence of a token. It will find out which cluster I am belongs to.

In case of supervised it will try out to find label when x, y, z features are given.

→ In that case first of all it will try to perform unsupervised learning inside it, it will try to perform certain task. But we need to divide data into diff-diff clusters.

In case of supervised learning, for score it will have some sort of vocabulary. From where it is taking one single-single words out of that.

• Un-supervised Model :-

→ In first step they have taken BPE (Byte pair Encoded) vocabulary.

Approximately 40000 vocabulary.

∴ For every kind of word it has some sort of embeddings (respectively numerical value).

- The model they have build:
That model will using 768 dimensional state for encoding token.
word embedding.
Whenever words you are going to insert it will try to represent it into 768 vector space.
- Word embedding + Positional embedding.
- 12 layer attention is being used.
(query, key and values)
- Optimizer was Adam Optimizer they have taken.
- Activation function wise they have used GELU.
GELU: Gaussian Error Linear Unit
- They have train entire model for 100 epochs. Batch size was 64.
- Batch wise in single batch they used 64 vectors.
- Sequence length was 512.
- In total model had 117 m parameters.

Objective of Supervised :- We have created diff-diff clusters. Just to fine tune or just to give verification that i have build a proper cluster.

- We need something for supervision.
- In ml task also we used to perform clustering task but by default it won't label those clusters. There labeling / verifying was done by human over there.

It's fine i have small amount of labeled data. Based on that i will be able to verify clusters based on that.

Final loss function:

$\mathcal{L} = \alpha \mathcal{L}_{\text{supervised}} + \beta \mathcal{L}_{\text{unsupervised}}$

It's combination of supervised loss + unsupervised loss.

→ Supervised learning model was just fine tuning the unsupervised one whether you are able to label something or able to create clusters in better way or not able to perform task in better way.

• Performance & Accuracy of Model:

→ GPT-1, it was state-of-art model

→ Out of 12 task of NLP, it was performing well with respect to 9 task.

It was powerful.

Here we have supervision of my supervised learning algorithms at top of my unsupervised, so that

if unsupervised is not going in right direction, it will be able to rectified, correct to it.

Based on this particular argument, this entire generative pre-training network is being build.

pre-training means, I have supervised model which is pre-trained model based on feature & models.

Based on that I am trying to build generative model which will try to generate something for me under supervision of pre-train model.

That is the meaning of Generative Pre-Training model.